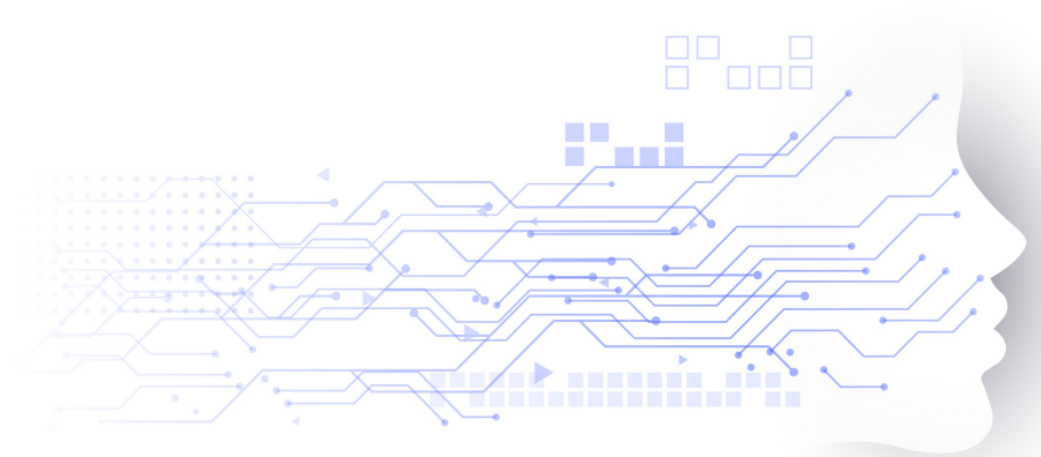




Evaluating Hypotheses

COSC2753 Machine Learning

Semester 2, 2026

Recap: Regression: a form of supervised learning

Experience: $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^n$ $\mathbf{x} \in \mathbb{R}^m$ $y \in \mathbb{R}$

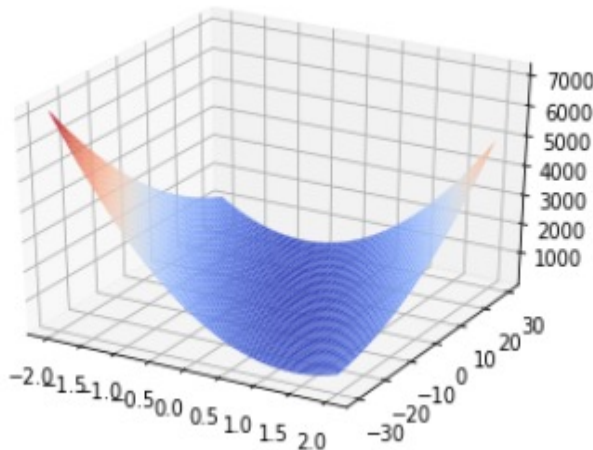
Hypothesis space:

- **Linear:** $h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m = \theta^{\top} \mathbf{x}$
- **Polynomial:**
$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \dots + \theta_i x_m + \theta_j x_m^2 + \dots$$

Loss function: $J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2$

Optimization: Gradient decent

$$\text{Repeat } \left\{ \begin{array}{l} \theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad \forall j \\ \end{array} \right.$$



Revision: Logistic Regression

A form of
Supervised learning

Experience: $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^n$ $\mathbf{x} \in \mathbb{R}^m$ $y \in \{0, 1\}$

Hypothesis space:

- Linear: $h_{\theta}(\mathbf{x}) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_m x_m) = g(\theta^{\top} \mathbf{x})$

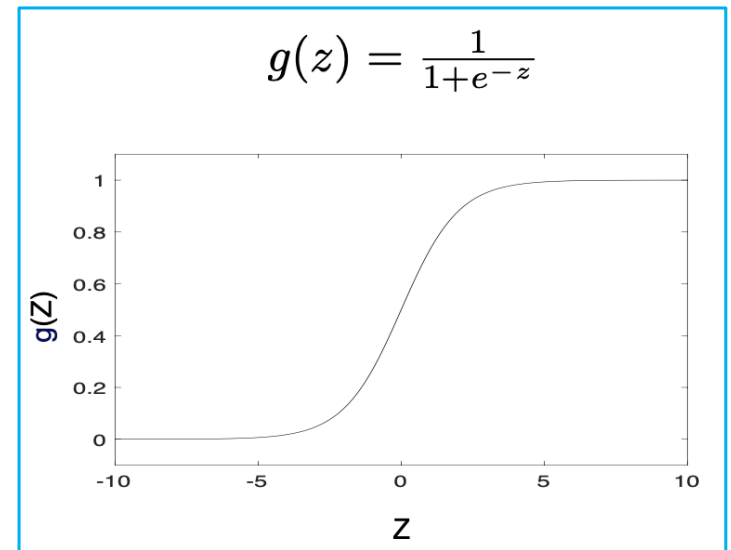
$$g(z) = \frac{1}{1+e^{-z}}$$

Loss function:

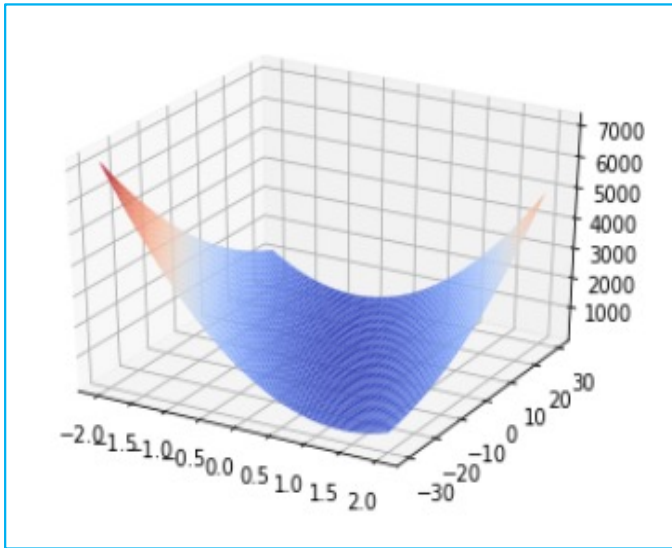
$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n -y^{(i)} \log(h_{\theta}(\mathbf{X}^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{X}^{(i)}))$$

Optimization: Gradient decent

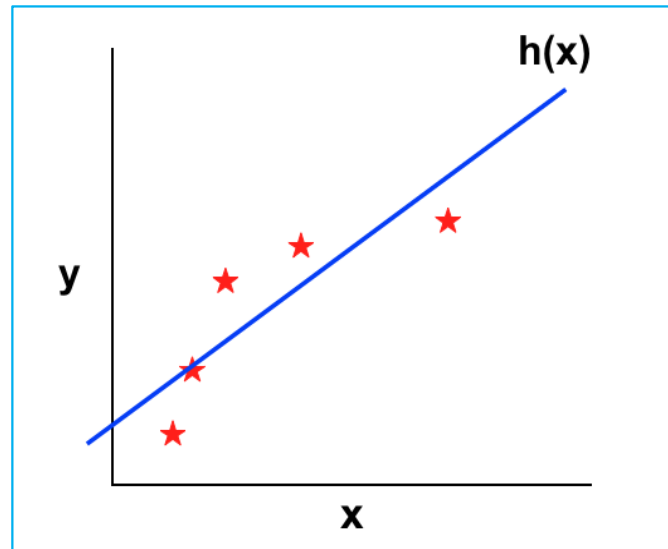
$$\text{Repeat } \left\{ \begin{array}{l} \theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad \forall j \\ \end{array} \right\}$$



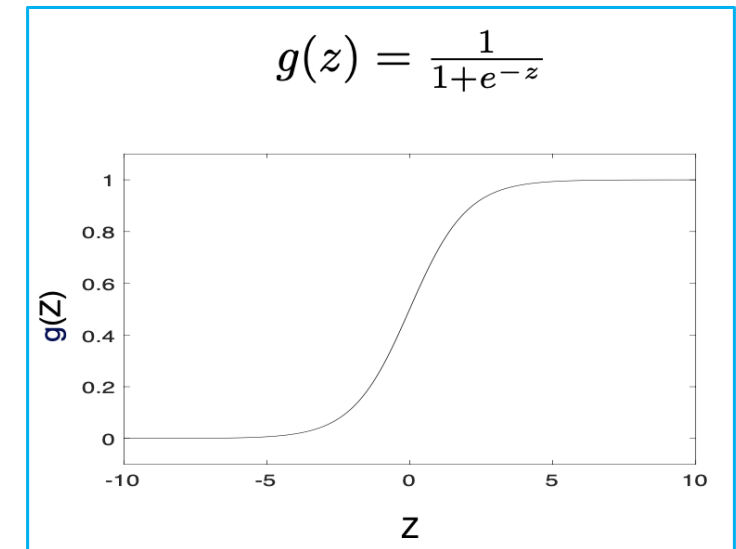
Revision: Which can be used for Regression?



Polynomial

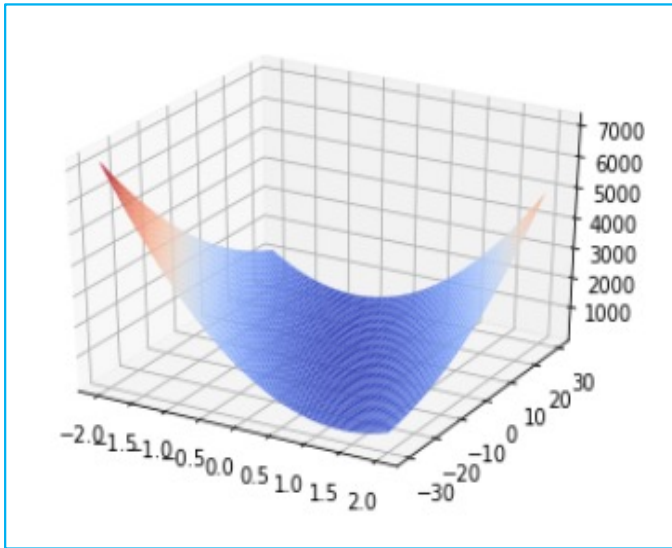


Linear

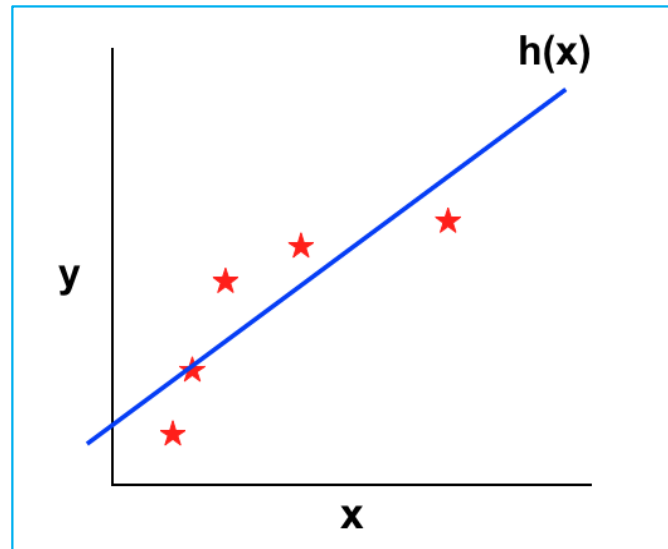


Logistic

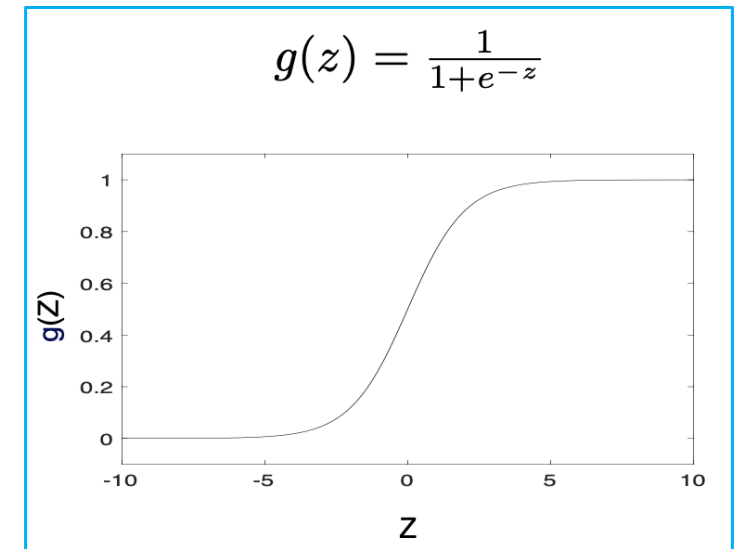
Revision: Regularization?



Polynomial



Linear

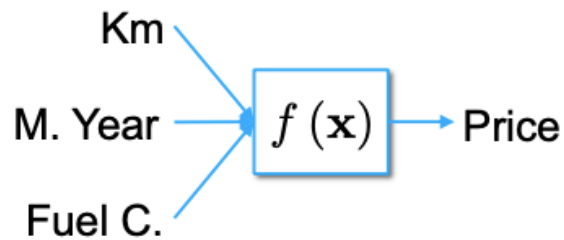


Logistic

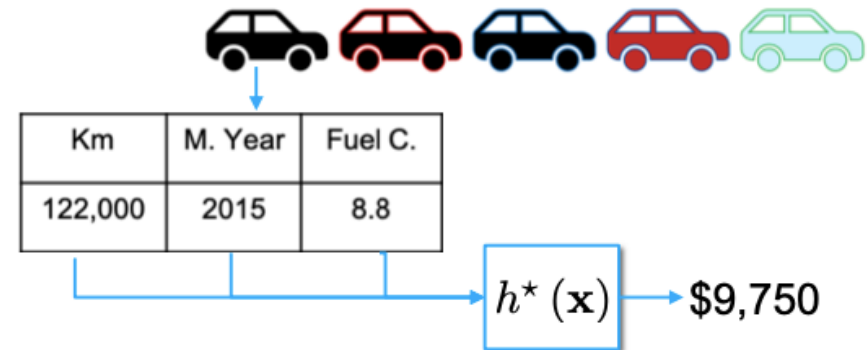
Measuring Performance

—

Model Development



Price	Km	M. Year	Fuel C.
10,000	102,000	2005	7.8
23,500	25,000	2010	5.2
12,250	256,000	2008	9.9
40,100	12,000	2018	11.2
5,000	23,000	2000	12.7
19,200	55,000	2015	12.4
12,500	121,000	2012	21.0



Assumption:

Nature of the relationship (e.g. Linear, polynomial)

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

Training:

Find the "Best hypothesis from the hypothesis space H "

e.g. :

$$h_{\theta}^*(\mathbf{x}) = 0.1 + 2.3x_1 + 1.1x_2 + .05x_3$$

Model development



- Did we use suitable attributes (features) for the problem?
- Are the assumptions valid?
 - e.g. The relationship between attributes and target variable linear.
- Will the derived hypothesis (model) **generalize well to new data**?
 - Has it overfitted to training data?
 - Is our assumption too limiting (under-fit)?



How good is the hypothesis (model)?

- We need an **evaluation framework** to measure performance of a hypothesis or model:
 - A **set of data** to measure the performance.
 - A **measure of “goodness”**— performance metric.



Independent test data

- To measure performance, **independent test data**:
 - Data that “**simulates**” unseen data
 - It mimics the process of using a hypothesis “for real”
 - Data which has **not been used for training**
- We will explore two mechanisms to generate a test set:
 - Hold-out validation
 - Cross-validation



Hold-out Validation

Constructing training/testing data



- Data set for Training and Testing are constructed by sub-dividing the experience (also called Hold-out Validation)
- Typically, an 80/20 split is used

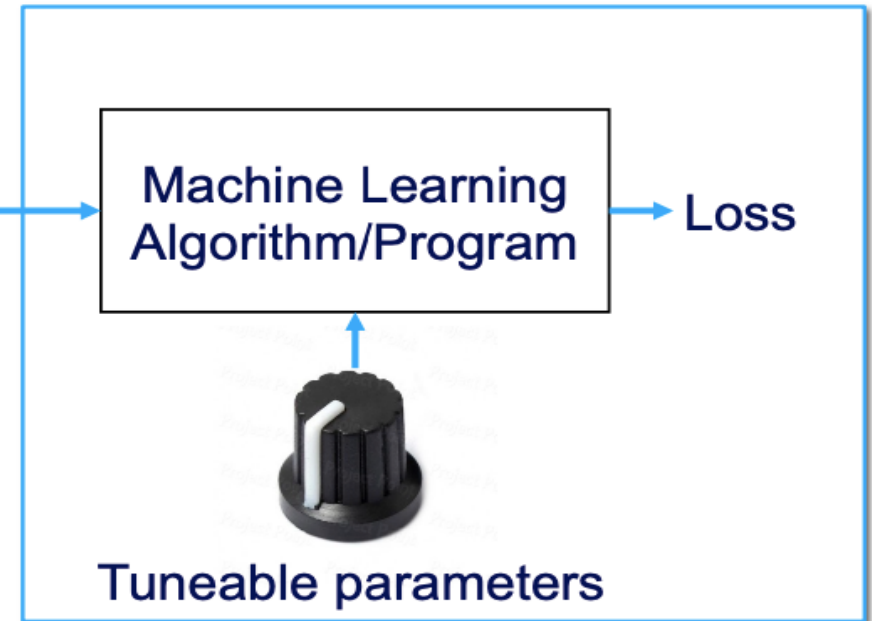


Machine Learning process

Stage 1: Find the optimal hypothesis from the hypothesis space

Price	Km	M. Year	Fuel C.
10,000	102,000	2005	7.8
23,500	25,000	2010	5.2
12,250	256,000	2008	9.9
40,100	12,000	2018	11.2
5,000	23,000	2000	12.7
19,200	55,000	2015	12.4
12,500	121,000	2012	21.0

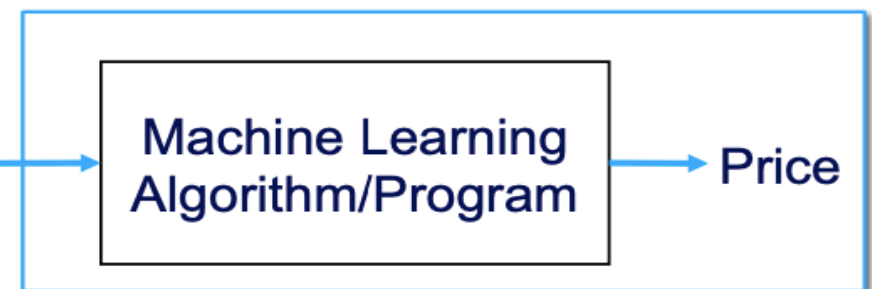
Training Data



Stage 2: Test the optimal hypothesis on “simulated” unseen data.

Km	M. Year	Fuel C.
122,000	2015	8.8

Test Data



Machine Learning Process

Stage 1.1: Find the optimal hypothesis from the hypothesis space **given a λ value**

Price	Km	M. Year	Fuel C.
10,000	102,000	2005	7.8
23,500	25,000	2010	5.2
12,250	256,000	2008	9.9
40,100	12,000	2018	11.2
5,000	23,000	2000	12.7
19,200	55,000	2015	12.4
12,500	121,000	2012	21.0

Training Data

Machine Learning
Algorithm/Program

Loss

Tuneable parameters

Stage 2: Test the optimal hypothesis on “simulated” unseen data.

Km	M. Year	Fuel C.
122,000	2015	8.8

Test Data

Best Hypothesis

Price

Stage 1.2: Find the best hypothesis amongst the hypotheses for **different λ value**

What data should we use in Stage 1.2?

What about hyper-parameters? How should we set them?

Hyper-parameter tuning



To measure performance, we require independent test data: Data that “simulates” unseen data.

It mimics the process of using a hypothesis “for real” data which has not been used for training (or testing!)

Once data has been used for testing, it is no longer independent

- Care must be taken to make limited use of the testing data
- Too much evaluation means testing data is being used to “train”

Think of test data as the exam.

You should never look at the exam questions while studying for the exam. If you do, you might end up only learning the thing that are on the exam.

Exam is not to measure how well you can memories the answers, it is about how well you know the concepts or skill.

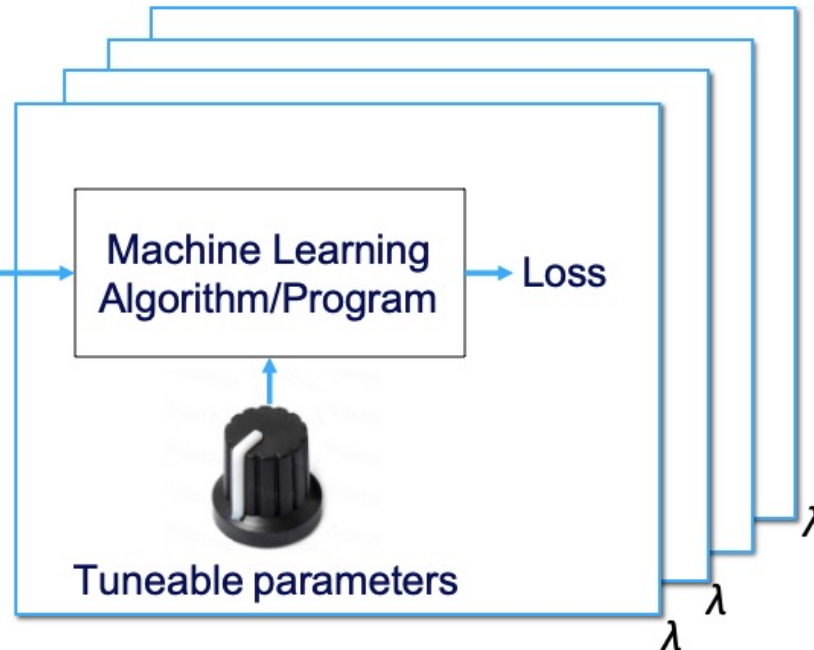


Machine Learning process

Stage 1.1: Find the optimal hypothesis from the hypothesis space **given a λ value**

Price	Km	M. Year	Fuel C.
10,000	102,000	2005	7.8
23,500	25,000	2010	5.2
12,250	256,000	2008	9.9
40,100	12,000	2018	11.2
5,000	23,000	2000	12.7
19,200	55,000	2015	12.4
12,500	121,000	2012	21.0

Training Data



Price	Km	M. Year	Fuel C.
10,000	102,000	2005	7.8
23,500	25,000	2010	5.2
12,250	256,000	2008	9.9

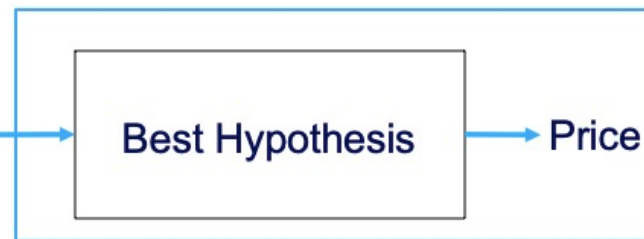
Validation Data

Stage 1.2: Find the best hypothesis amongst the hypotheses for **different λ value**

Stage 2: Test the optimal hypothesis on “simulated” unseen data.

Km	M. Year	Fuel C.
122,000	2015	8.8

Test Data



Constructing training/testing data



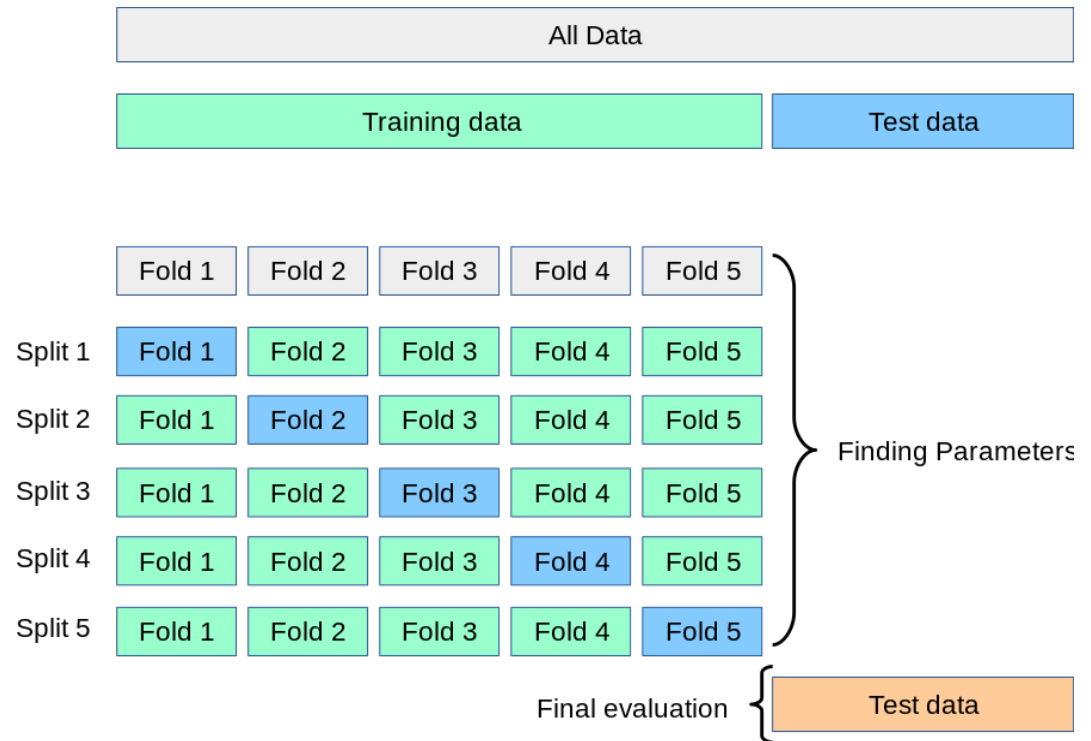
- Typically, a data set is sub-divided into three sets:
 - Training data - for training a hypothesis
 - Validation data - for “testing” and tuning parameters
 - Testing data - for evaluating and comparing **final hypotheses**
- Typically, a 60/20/20 split



Cross Validation

K-fold Cross Validation

- Divide data into k partitions
- One partition: validation set, all other $k - 1$: form training set
- Evaluate then repeat: different partitions as validation set
- Average results



K-fold Cross Validation



- Note:
 - K-fold cross validation gives a measure of the average expected error of our ML
 - BUT, you still need to **choose ONE hypothesis** (model)

- How to choose a model?????



K-fold Cross Validation



- Note:
 - K-fold cross validation gives a measure of the average expected error of our ML
 - BUT, you still need to **choose ONE hypothesis** (model)
- How to choose a model?
 - A simple method is to choose a model **which gives an error which is similar to the average error**
 - However, this model may still be biased
 - OR, use techniques such as regularisation to “reduce a model complexity” to an error similar to the average

Evaluation Metrics

Evaluation metrics



- Why not use the loss function?
 - Loss function: make optimization process easy.
The value of the loss function: not be very intuitive.

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n -y^{(i)} \log(h_{\theta}(\mathbf{X}^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(\mathbf{X}^{(i)}))$$

- Evaluation metric is selected so that they are intuitive.
- 

Regression Evaluation Measures

Regression evaluation metrics

Mean absolute error:

$$\text{MAE} = \frac{1}{\bar{n}} \sum_{i=1}^{\bar{n}} |h(x^{(i)}) - y^{(i)}|$$

Mean squared error:

$$\text{MSE} = \frac{1}{\bar{n}} \sum_{i=1}^{\bar{n}} (h(x^{(i)}) - y^{(i)})^2$$

Root mean squared error:

$$\text{RMSE} = \sqrt{\frac{1}{\bar{n}} \sum_{i=1}^{\bar{n}} (h(x^{(i)}) - y^{(i)})^2}$$



What differences between them?

Regression evaluation metrics



- **Mean absolute error (MAE):**

$$MAE = \frac{1}{n} \sum_{i=1}^n |h(x^{(i)}) - y^{(i)}|$$

- **Mean squared error (MSE):**

$$MSE = \frac{1}{n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2$$

- **Root mean squared error (RMSE):**

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (h(x^{(i)}) - y^{(i)})^2}$$

- MSE amplifies the effect of outliers (or errors in a single instance) compared to MAE.
- Conversely, MAE dilutes the impact of single instances as the data set increases

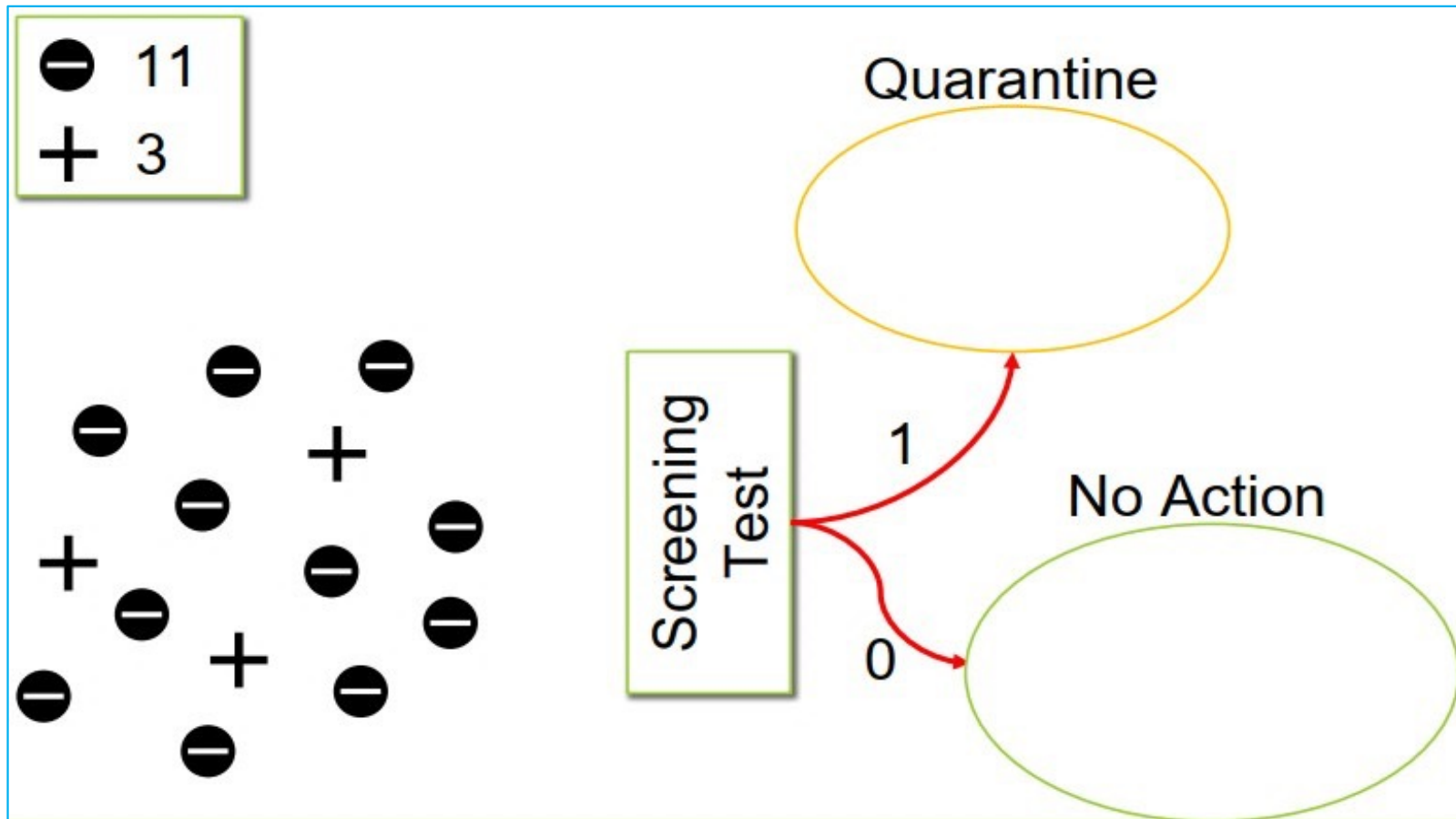
Classification Evaluation Measures

Classification evaluation metrics



- Evaluate how well the classifier $h(x)$ performs in comparison to actual classes
- With classification, different types of metrics
 - Confusion Matrix
 - Accuracy, Precision, Recall, F1-Score
 - ROC curve

Example: Screening COVID-19 @ airport

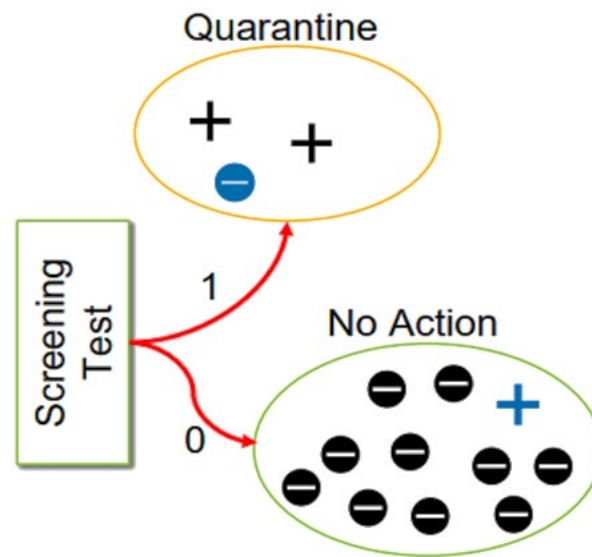


Example: Screening COVID-19 @ airport

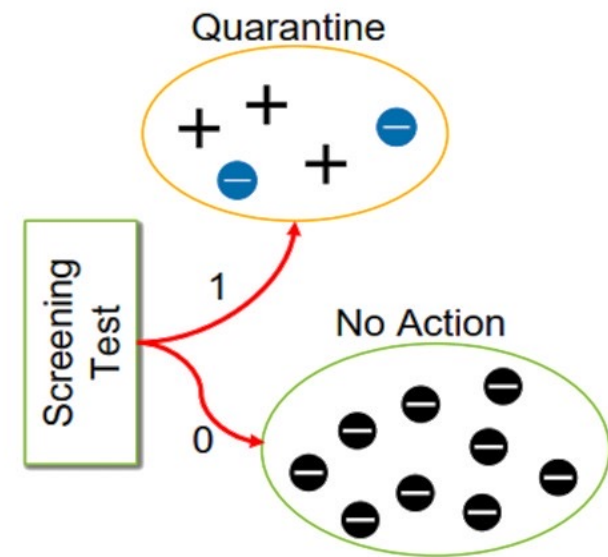
Screening done at airport:

- False Positive (Type I): **Non-COVID** patients **detected** as COVID by test.
- False Negative (Type II): **True COVID** patient **not detected** by system.

⊖	11
+	3

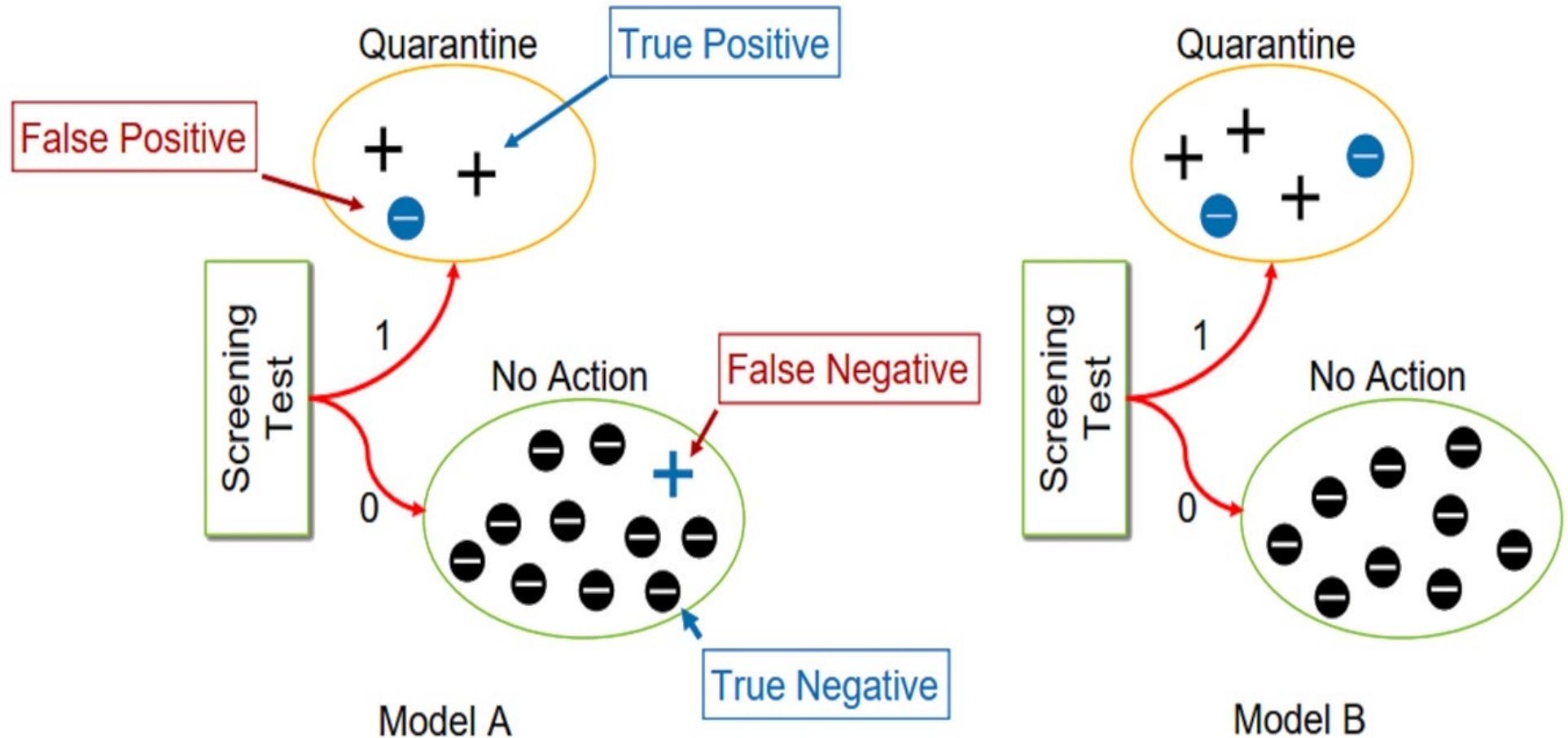


Model A



Model B

Example: Screening COVID-19 @ airport



Classification errors (Type 1 & 2)



- Consider binary class problems
- Assume there are some classification errors
 - Predict one class, but actually the test data had the other class
- Not all classification errors are the same
- From the COVID-19 example:
 - Patient does not have COVID-19 but received a positive diagnosis (Type 1 error)
 - Patient does have COVID-19 but received a negative diagnosis (Type 2 error)

Types of classification outputs

- True Positive (TP): class 1 predicted as class 1
- False Positive (FP): class 0 predicted as class 1 (type 1 error)
- True Negative (TN): class 0 predicted as class 0
- False Negative (FN): class 1 predicted as class 0 (type 2 error)

Total number of instances

$$m = TP + FP + TN + FN$$

		Prediction	
		Positive	Negative
Actual	Positive	True Positive	False Negative
	Negative	False Positive	True Negative

Confusion matrix



- Summarise the four types of classification error

		Predicted		Y	h(X)	
		1	0			
Actual	1	TP	FN	1	1	TP
	0	FP	TN	0	0	TN
				0	0	TN
				0	1	FP
				1	1	TP
				0	0	TN
				0	1	FP
				1	0	FN
				1	1	TP

$TP = 3$
 $TN = 3$
 $FN = 1$
 $FP = 2$

		Predicted	
		1	0
Actual	1	3	1
	0	2	3

Common measures: Accuracy

Accuracy measures the rate of **correct classifications**

$$\frac{TP + TN}{TP + FP + FN + TN}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Common measures: Precision

Precision measures:

- Of all the positive instances that were identified, what ratio are actually positive?
- Is the classifier **good at being correct when it identifies a positive instance?**

$$\frac{TP}{TP + FP}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Common measures: Recall

Recall measures:

- Of all the positive instances, how many does the classifier get correct?
- Also called, the **true positive rate**, or **sensitivity**

$$\frac{TP}{TP + FN}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Common measures: Specificity



Specificity measures the “inverse” of recall

- Also known as the **true negative rate**
- Of all the actual negative instances, how many does the classifier get correct?

$$\frac{TN}{TN + FP}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Common measures: F1 Score

The **F1 score**, is the harmonic average of precision and recall

- An F1 score of 1 is perfect precision & recall
- Provides a balance between precision & recall
- More reliable measure than accuracy if the confusion matrix is skewed

$$2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

		Predicted	
		T (1)	F (0)
Actual	T (1)		
	F (0)		

Class balance



Assume that you have a binary classification problem (cancer vs healthy) where the data-set has the following distribution:

- Class 0 (Healthy) : 1000 examples
- Class 1 (Cancer) : 25 examples
- What will be the accuracy if you have a trivial model that always says healthy?

$$\text{Acc} = 1000/(1000+25) = 97.5\%$$

- Can we use this model?



ROC Curve



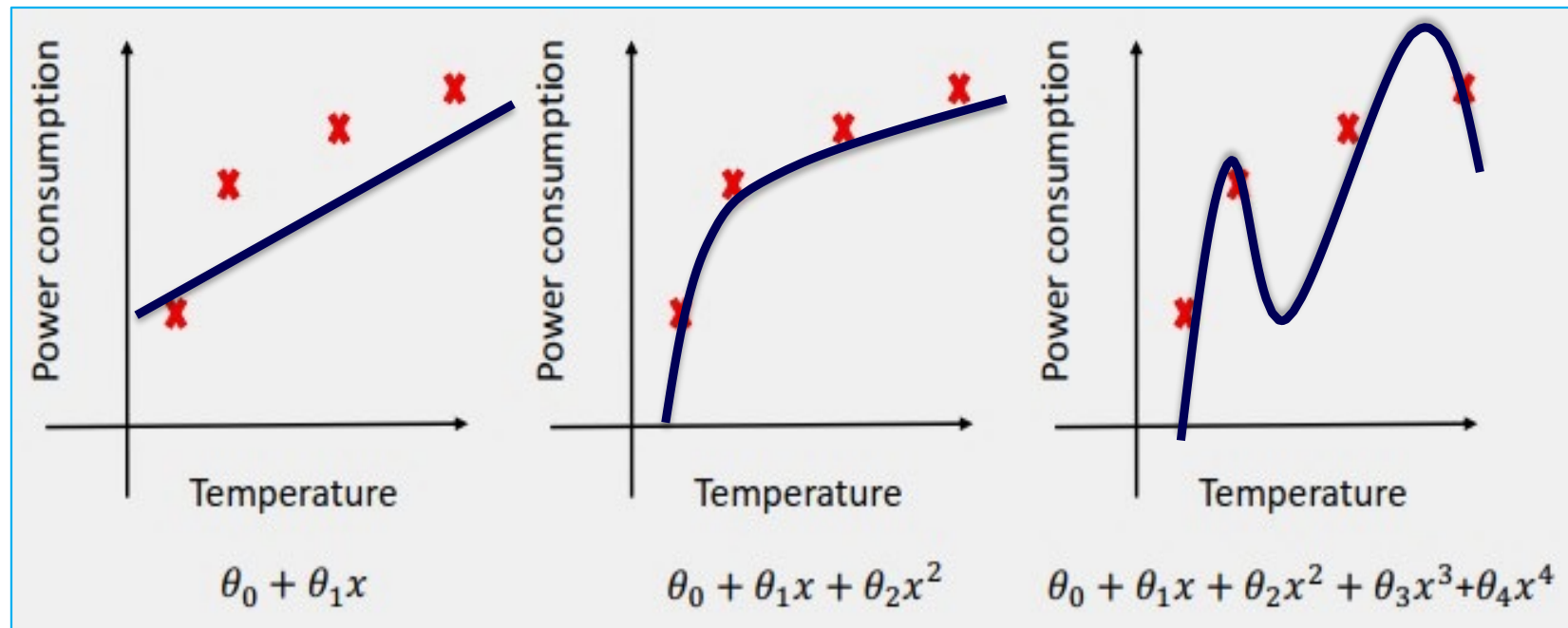
Plots the recall (true positive rate) against the false positive rate
A useful tool for visualising the performance of a classifier

<https://www.youtube.com/watch?v=4jRBRDbJemM>

Hyper-parameter Tuning

Problem of Over-fitting and Under-fitting

Aim of machine learning is to learn **generalised models** to be good at predicting future, unseen input data



under-fit

just right

over-fit

Over-fitting & Under-fitting

Over-fitting can occur if we have too many features

The learned hypothesis may fit the training data too well

But may not generalise to new examples (e.g., predict energy usage on new examples)

Under-fitting can occur if we have too few features

The learned hypothesis is not able to fit the data well

Is unable to make accurate predictions

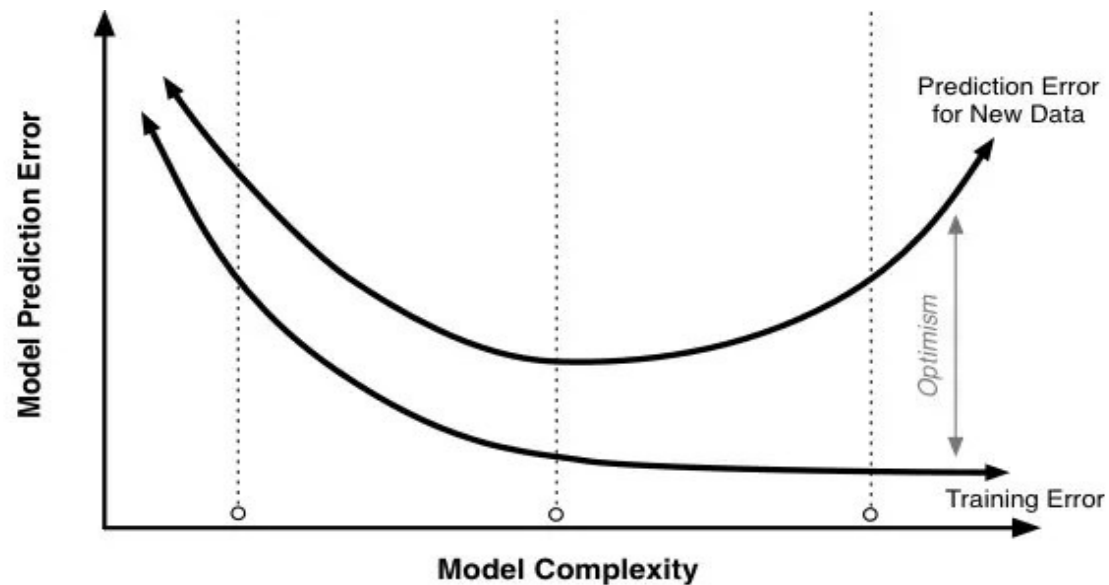


How to identify over/under-fitting

Assume data split into train and validation sets

$$J_{train}(X) = \sum_{training} L(h(X), Y) \quad (\text{training error})$$

$$J_{valid}(X) = \sum_{validation} L(h(X), Y) \quad (\text{test/validation error})$$

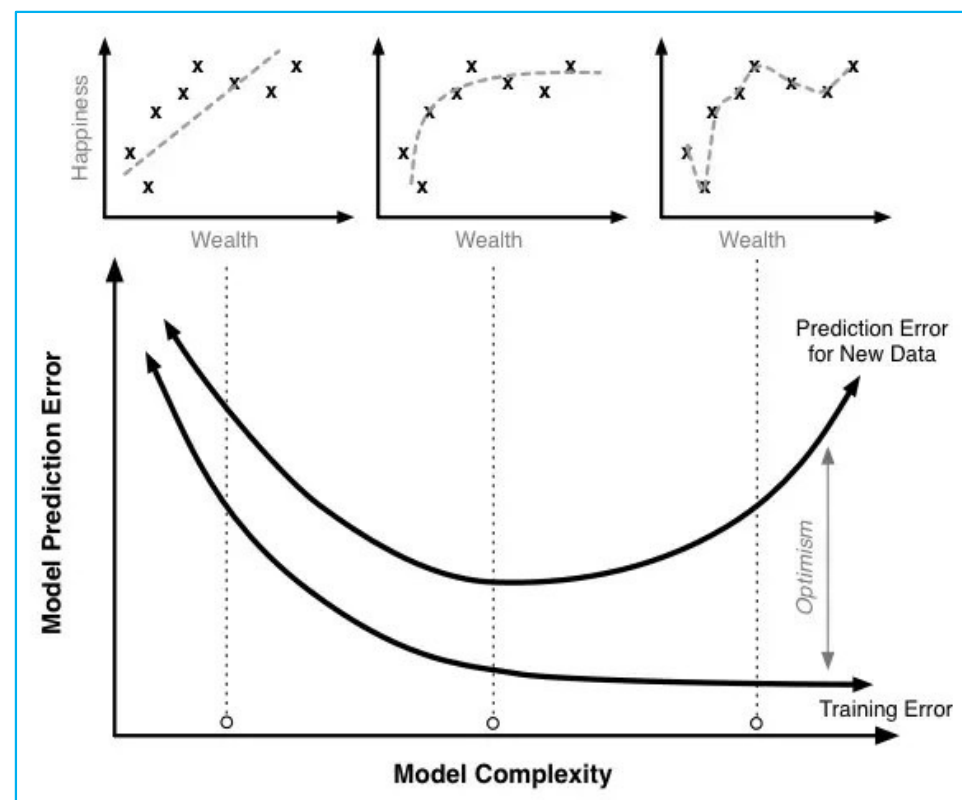


Over-fitting & Under-fitting

Change λ and observe the training and **validation** errors.

- Pick the λ^* value that has the smallest gap and best validation performance.
- The best hypothesis will be the optimal hypothesis at λ^*

Never use test data to do this.
Always use validation data or
cross validation.



Revision

- Metrics for Regression
 - MAE, MSE, RMSE, R^2
- Metrics for Classification
 - Accuracy, Precision, Recall, F1
 - Confusion Matrix
- Validation methods
- Hyper-parameter fine-tuning



Thank you!